TU Berlin
Institute of Mathematics
Dr. Peter Münch

0. Homework set

Deadline: Sun, May 3 2026

Introduction

There will be 6+1 homework sets in the course *FEM for the Navier–Stokes Equations*. They will comprise different types of tasks, which are labeled according to the following legend:

- information,
- reading task: classical papers,
- theoretical considerations,
- perform experiment,
- install software,
- programming tasks.

To receive credits, you are expected to write a short report individually (0. Homework set) or as a group by the deadline indicated at the top of the homework set. Please send the report to

muench@math.tu-berlin.de.

In this homework set, you will review the basics of finite-element computations (Problem 0.1) and install the necessary software (Problems 0.2–0.4). Individual tasks have individual "deadlines" since we will use their outcome on different days. Nevertheless, please submit the report as a single report. Use LaTeX for writing the report.

Problem 0.1: Finite-element computation

In the course "Numerical methods for PDEs", we developed a simple Python-based code for the solution of the Poisson problem with the finite element method

$$\begin{aligned} -\Delta u &= f(\vec{x}) && \text{in } \Omega, \\ u &= g(\vec{x}) && \text{on } \Gamma. \end{aligned}$$

Its overall structure is dimension-independent (i.e., the same code handles both 1D and 2D problems).

In 2D, unstructured meshes consisting only of triangles (see Figure 1) are generated by `meshpy`.

Three scenarios are implemented:

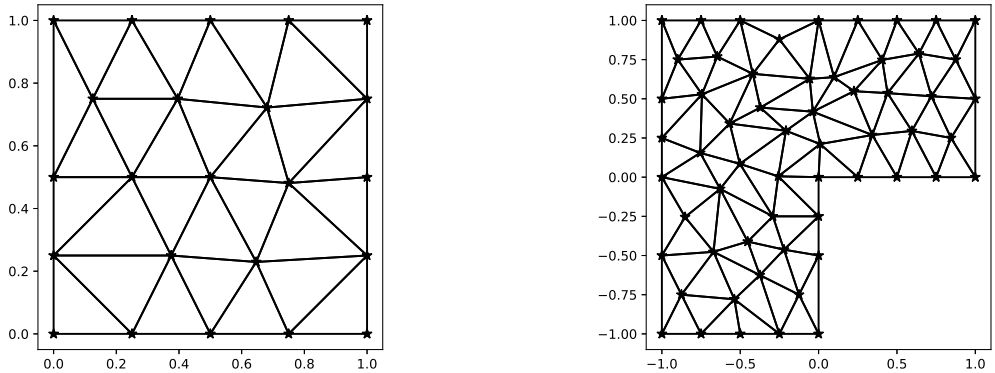

Figure 1: Implemented unstructured meshes: hypercube (left) and L-shaped domain (right).

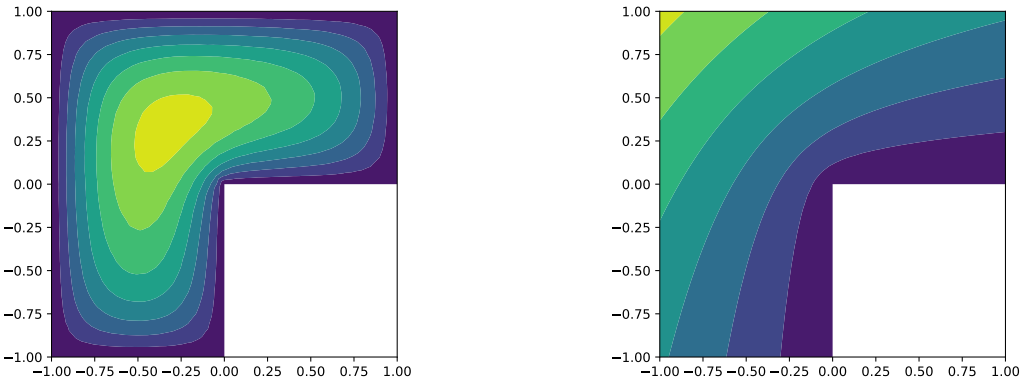

Figure 2: Expected solutions of the 2D Poisson problem on L-shaped domain.

a) Solve the Poisson equation on a 1D unit domain with inhomogenous Dirichlet boundary conditions and the source term:

$$g(x) = \exp(-10(x-0.5)^2),$$
$$f(x) = 20(1-20(x-0.5)^2)\exp(-10(x-0.5)^2).$$

The solution $u(\vec{x}) = g(\vec{x})$ is analytically given so that we can compute the error.

b) Solve the Poisson equation on an L-shaped domain with homogenous Dirichlet boundary conditions and unit source term:

$$g(\vec{x}) = 0,$$
$$f(\vec{x}) = 1.$$

Unfortunately, there is no analytical solution. The left panel of Figure 2 shows the solution that you can expect.

c) Solve the Poisson equation on an L-shaped domain with inhomogenous Dirichlet boundary

conditions (given in polar coordinate system) without source term:

$$g(r,\,\phi) = r^{2/3}\sin(2/3\phi),$$
$$f(r,\,\phi) = 0.$$

The solution $u(\vec{x}) = g(\vec{x})$ is analytically given so that we can compute the error. The right panel of Figure 2 shows the solution that you can expect.

Read and understand the code (`HW_00_code_fem.tar.gz`.). Run it and vary the parameters; in particular, perform a convergence study in the L_2 norm for a) and c) by refining the mesh. Is the convergence rate what you expect for linear finite elements? If not, explain why.

In the report, describe the structure of the code and present the results of the convergence study.

Deadline: Thu, 16.4 (We will discuss this task in class.)

🖫 Problem 0.2: Installation: terminal

In this course, you will heavily use the terminal. On Linux (a recent version of Ubuntu is recommended) and macOS, you can use the native terminal. On Windows, install Ubuntu using a Windows Subsystem for Linux (WSL).

In the report, shortly describe the operating system and the terminal/shell you are using.

Deadline: Mon, 20.4 (We will use the terminal in class.)

🖫 Problem 0.3: Installation: software

Install the necessary software:

a) a `C` and `C++` compiler, e.g.: `gcc` or `clang`,

b) `CMake` for building projects,

c) `Doxygen` for documentation of your C++ code,

d) `git` for version control,

e) `Gmsh` for generating computational meshes,

f) `ParaView` for postprocessing simulation results,

g) an IDE (Integrated Development Environment) of your choice, e.g.: `VSCode`.

You may use a package manager (e.g., `apt`, `brew`, etc.).

In the report, briefly describe the steps/commands you took to install each software. If the software was preinstalled on your system, state this in the report.

Deadline: Mon, April 20 (We will use these tools in class.)

🖫 Problem 0.4: Installation: deal.II

Install the finite-element library `deal.II`.

In the following, two ways of installing deal.II on Ubuntu (and Windows via WSL) are shown:

a) install `deal.II` via package manager (takes minutes): the following commands use apt-get to install `deal.II` including all dependencies:

```
export REPO=ppa:ginggs/deal.ii-9.6.0-backports

sudo apt-get update && apt-get install -y software-properties-common
sudo add-apt-repository $REPO
sudo apt-get update
sudo apt-get install libdeal.ii-dev
```

To test the installed `deal.II` package, a toy code (`HW_00_code_dealii.tar.gz`; a simplified version of step-1 from the `deal.II` tutorials) has been provided online. Unpack it and run it with:

```
cd HW_00_code_dealii
mkdir -p build
cd build
cmake ..
make

./step-1
```

Two .svg files(vector graphics) are created in your current directory (you can see them with `ls`), which you can view, e.g., in a browser.

b) install `deal.II` from scratch (takes about one hour): the following commands download the program `candi` and install `deal.II` (including all the dependencies; in particular, `Trilinos` and `p4est`) with it to a prescribed place:

```
git clone https://github.com/dealii/candi.git
cd candi

sed -i 's/^#\?DEAL_II_VERSION=.*/DEAL_II_VERSION=v9.6.2/' candi.cfg

./candi.sh \
        --prefix="$HOME/software/dealii-9.6/" \
        --packages="p4est trilinos dealii" \
        --jobs=4
```

For testing purposes, we run the same program as before, except that we explicitly specify the path to `deal.II`:

```
cd HW_00_code_dealii
mkdir -p build
cd build
cmake -DDEAL_II_DIR="$HOME/software/dealii-9.6/deal.II-v9.6.2/" ..
make

./step-1
```

For macOS, packages are provided on

`https://github.com/dealii/dealii/releases/tag/v9.6.2`.

Alternatively, you can use `candi`. For more details on installing deal.II, see

`https://github.com/dealii/dealii/wiki/Getting-deal.II`.

In the report, present the steps/commands you took to install `deal.II`. Add the created .svg files to the report.

Deadline: Mon, 27.4 (We will run and analyze `deal.II` code in class.)